AI Cost Autopilot: A Compiler-Aware Gateway for Token-Cost Optimisation in LLM Coding Agents

Project Proposal for UCS503P
Thapar Institute of Engineering and Technology

Agampreet Kaur, CSED* Devansh Sharma, CSED† Furmaandeep Kaur, CSED‡

August 31, 2026

1 Higher-order goal

This project aims to make AI-assisted software development economically sustainable by reducing the computation paid for on every request to a Large Language Model (LLM). While the topic is broader than any single development team, the immediate engineering objective is to translate *efficient context usage* into a measurable, deployable piece of software: a gateway that sits between a coding agent and the model it calls.

2 Time-to-value

- We shall deliver meaningful increments quickly: a transparent pass-through gateway first, then one optimisation at a time, each measurable on its own.
- We shall prioritise fast feedback over perfection, using weekly iterations with CI-backed automation from the first week to reduce release friction.
- Success is defined by what can be delivered and *measured* in the first iteration, then improved — every subsequent component must demonstrate a token reduction that the previous configuration did not already achieve.

3 Problem Statement

Developers increasingly work through LLM coding agents such as Cursor, Claude Code and Codex. These tools operate by transmitting source code and instructions to a hosted model, which bills per *token*. As a codebase and a team grow, both the size and the number of these requests grow with them. Common pain points include:

- **Bulk context transmission:** agents attach large portions of a repository to every request, including files the task does not need.
- **Repetition:** semantically identical questions are re-asked and re-billed, because nothing remembers previous answers.

*Roll No: 1024240033, Email: akaur11_be24@thapar.edu
†Roll No: 1024240012, Email: dsharma10_be24@thapar.edu
‡Roll No: 1024240029, Email: fsaini_be24@thapar.edu

- **Budget unpredictability:** spend scales with usage in a way that teams cannot forecast or cap.
- **Exposure of private code:** more source leaves the developer's machine than the task requires.
- **Unsafe existing remedies:** generic prompt compressors treat code as plain text and may delete a line of logic the model needed, trading correctness for savings.

Why this matters now (contextual relevance): adoption of coding agents is rising faster than any discipline for controlling their cost, and no standard layer exists between the developer's tool and the model to manage it.

4 Proposed Solution

4.1 Overview

Build a **provider-agnostic gateway** that intercepts each request and reduces its cost before forwarding it, with the following components:

- **Gateway:** an OpenAI/Anthropic-compatible endpoint, so an existing tool connects by changing a single base-URL setting.
- **Compiler-aware trimmer:** removes code the task does not need, using the structure of the code rather than its text.
- **Semantic cache:** reuses a stored answer when a new request means the same thing as an earlier one.
- **Autopilot policy:** selects, per request, the cheapest action that is still safe.
- **Metering:** records tokens before and after, cost, and cache hit-rate for every request.

4.2 Core workflow

1. The coding agent issues a request to the gateway instead of the provider.
2. The gateway computes cheap signals (token count, presence of code) and the autopilot selects a plan.
3. If a semantically equivalent request was answered before, the stored answer is returned without contacting the model.
4. Otherwise the trimmer reduces the request, preserving the code the task depends on.
5. The reduced request is forwarded to the provider; the answer is returned and recorded against the *original* request.

4.3 Operational constraints for an educational setting

- The system must run on student laptops: no GPU, no cluster.
- Every optimisation must degrade gracefully — if an optional dependency is absent, the gateway continues to work as a pass-through.
- We avoid claims of novel research; the contribution is engineering judgement, measurement discipline and safety, not a new algorithm.

5 Solution Approach

5.1 Context reduction

Reduction is deterministic and structural, not statistical. Source files are parsed into an Abstract Syntax Tree; a call graph is derived; the function the task concerns and the functions it depends on are retained in full, while every other definition is reduced to its signature. Because edits

are made at syntactic boundaries, the reduced file remains valid source. Where a dependency cannot be resolved with confidence, the approximation deliberately retains *more* than necessary: an unnecessary retention costs savings, whereas an incorrect removal costs correctness.

5.2 System architecture

A three-tier arrangement:

- **Interface:** an HTTP endpoint mirroring the provider API, so no client modification is required; streaming and tool-call structures pass through unchanged.
- **Optimisation layer:** the trimmer, the cache and the policy, each an independent module behind a fixed interface so they can be developed and tested in isolation.
- **Provider adapter:** forwards to the upstream model, with an offline mock so the whole pipeline can be exercised without an API key.

5.3 CI/CD and fast delivery enabler

Continuous integration runs the full test suite on every change, so a modification to one module cannot silently break another. Every change reaches the main branch through a reviewed pull request. Project documentation is built and published automatically from the repository.

6 Evaluation Criterion

6.1 Primary evaluation metric

Incremental token reduction at equal task outcome: the percentage of input tokens removed by our system *beyond* what a naive alternative already achieves, on requests where the agent still completes the task correctly.

- Target: $\geq 30\%$ reduction against an untouched request, on code-heavy repetitive workloads.
- Attribution: measured with a tokeniser on the exact payloads sent, logged per request by the gateway itself.

6.2 Secondary evaluation metrics

- **Behavioural equivalence:** the agent reaches the same outcome with and without the gateway, over repeated runs. A saving that changes the result is counted as a failure, not a saving.
- **Cache hit-rate** and the proportion of cost avoided by reuse.
- **Added latency:** time introduced by the gateway per request.
- **Context exposure:** how much source code leaves the machine, relative to the unmodified agent.

6.3 Validation plan

A single fixed workload is run through progressively more capable configurations, and the *incremental* contribution of each is reported: (i) the untouched request, as a control; (ii) trivial textual cleanup; (iii) generic text compression; (iv) our compiler-aware reduction; (v) with semantic reuse enabled. This ablation is deliberate: a single percentage quoted without a baseline cannot distinguish our contribution from what a trivial transformation would have achieved anyway. Each row is reported alongside its behavioural-equivalence result, so cost and correctness are never presented separately.

7 Scalability

1. **Scalable (theoretically):** the gateway holds no per-request state, so instances may be added independently; cached entries are partitioned by request settings so lookups do not degrade as unrelated traffic grows; the store is addressed through an interface that admits a dedicated vector database.
2. **Operationally deployable (practically):** it runs as a single local process for one developer, or as a shared instance for a team, where a common cache increases the proportion of reused answers.

8 Engine availability heuristic

Mature components exist for every part of this system, so effort can be spent on design and measurement rather than infrastructure:

- Web framework for an HTTP API, and an HTTP client for forwarding.
- Incremental parser with grammars for mainstream languages.
- Tokeniser for measurement, matching provider billing.
- Sentence-embedding model and a vector store for semantic reuse.
- Test framework, continuous-integration runner and documentation publishing.

9 Project scope and deliverables

9.1 Initial deliverable

- Gateway forwarding requests to a provider, with token and cost logging.
- Structural reduction for one language, preserving valid syntax.
- Exact-match reuse of previously answered requests.
- A policy selecting between reuse, reduction and pass-through.
- CI running the test suite on every change; published documentation.

9.2 Subsequent deliverables

- Dependency-aware retention across multiple files.
- Semantic reuse with a calibrated similarity threshold.
- A second language grammar.
- The ablation benchmark and the measured results it produces.

10 Risks and mitigations

- **Reduction removes needed context:** retain the task's dependencies in full, bias every approximation towards retention, and fall back to pass-through when a file cannot be parsed.
- **Reuse returns a plausible but wrong answer:** gate reuse behind a conservative similarity threshold calibrated on source code rather than prose, and never reuse across differing model or request settings.
- **Savings are indistinguishable from a trivial baseline:** report incremental gains against explicit baselines rather than a single headline figure.
- **Provider interfaces change:** isolate provider specifics behind a thin adapter.
- **Scope creep:** one language and one provider first; additional coverage only after the first iteration is measured.

11 Summary

This project proposes a deployable gateway that reduces the token cost of LLM coding agents without changing developer workflow or agent behaviour. It meets the course criteria by emphasising time-to-value through weekly measurable increments, using CI/CD to support frequent safe releases, and defining evaluation criteria that are both measurable and attributable — specifically, incremental token reduction reported against explicit baselines and paired with behavioural equivalence. It remains focused on engineering workflow, safety and measurement rather than research novelty.